UNIVERSIDAD NACIONAL DE INGENIERÍA — Facultad de Ingeniería Industrial y de Sistemas — Unidad de Posgrado

Laboratorio 12b: SRGAN desde cero

Laboratorio de programación

Redes Neuronales y Aprendizaje Profundo

Aldo Camargo Fernández-Baca, Ph.D.

Clase asociada	Clase 12: Superresolución con GANs (SRGAN)
Modalidad	Individual o en parejas
Duración	**2 horas** (máximo)
Entorno	Google Colab con GPU (T4 o superior)
Material	`Lab12b_SRGAN_Programacion.ipynb` (notebook de inicio con pruebas automáticas)
Fecha de entrega	_____
Puntaje total	20 puntos

Objetivo

Implementar **con sus propias manos** todos los componentes de SRGAN (Ledig et al., 2017): el *pixel shuffle*, la inicialización ICNR, el generador SRResNet, el discriminador, la pérdida VGG, los pasos de entrenamiento con MSE y adversarial, la interpolación de redes de ESRGAN y una métrica de parpadeo temporal para video. Al final entrenarán su propia SRResNet y su propia SRGAN y superresolverán un video.

1. Cómo funciona este laboratorio

- Abra `Lab12b_SRGAN_Programacion.ipynb` en Colab con GPU y ejecute la *Preparación* (utilidades, datos de Flowers-102, funciones auxiliares). No la modifique.
- Cada tarea tiene una celda con funciones o clases incompletas (`raise NotImplementedError` o `# TODO`) y, debajo, una celda `verificar_k()` con **pruebas automáticas**. Si su código es correcto, verá ✓. **No modifique las celdas de verificación.**
- Las especificaciones exactas (formas, número de parámetros, orden de las operaciones) están en el notebook. Las pruebas comprueban, por ejemplo, que la SRResNet tenga exactamente 1 549 462 parámetros y el discriminador 23 565 505.
- Las Tareas 6 y 7 terminan con un entrenamiento dado (5 000 y 3 000 iteraciones, unos minutos cada uno en una T4). **Mientras entrenan, avance con las Tareas 8 y 9**, que no dependen de la GPU para sus pruebas.
- Puede consultar las notas de la Clase 12. No copie el código del notebook de clase: las pruebas son las mismas para todos, pero la nota de "código limpio" revisa que el código sea suyo y esté comentado.

2. Tareas

Tarea 1 *Pixel shuffle* a mano [1.5 pts]

`pixel_shuffle_manual(t, r)`: $(N, Cr^2, H, W) \to (N, C, rH, rW)$ con $\mathcal{PS}(T)_{n,c,y,x} =$

$T_{n,\,cr^2+r(y \bmod r)+(x \bmod r),\,\lfloor y/r\rfloor,\,\lfloor x/r\rfloor}$, usando solo `view`/`reshape` y `permute`. Prohibido `nn.PixelShuffle` y `F.pixel_shuffle` (la prueba lo revisa).

Tarea 2 Inicialización ICNR [1.5 pts]

`icnr_(conv, r)`: inicializar C filtros con Kaiming y repetirlos en los r^2 canales consecutivos de cada grupo (pesos y sesgo). La prueba verifica que $\mathcal{PS}(\text{conv}(x))$ sea igual a "convolución con C filtros + vecino más cercano".

Tarea 3 Generador SRResNet [3 pts]

`BloqueResidual`, `BloqueSubpixel` y `SRResNet(B=16, c=64, escala=4)`: conv 9×9 + PReLU; B bloques residuales (conv–BN–PReLU–conv–BN + salto); conv 3×3 + BN + **salto global**; $\log_2$(escala) bloques sub-píxel (con ICNR); conv 9×9 y `tanh`. Las pruebas revisan formas para escalas 2, 4 y 8, el número de parámetros y la presencia del salto global.

Tarea 4 Discriminador [2 pts]

Ocho convoluciones 3×3 (pasos $1, 2, 1, 2, \ldots$; canales $64 \to 512$), BN en todas menos la primera, LeakyReLU(0.2), `AdaptiveAvgPool2d(6)`, densas 1024 y 1. Devuelve un **logit**: sin sigmoide.

Tarea 5 Pérdida VGG [2 pts]

`PerdidaVGG('22'|'54')`: VGG19 de `torchvision` truncada justo después de `relu2_2` o `relu5_4` (ustedes deben encontrar los índices), congelada y siempre en modo `eval`; convierte $[-1, 1] \to [0, 1]$, normaliza con la media y desviación de ImageNet y devuelve la MSE entre características. `hr` no debe recibir gradiente.

Tarea 6 Paso de SRResNet [2 pts]

`paso_srresnet(G, opt, lr, hr)`: un paso con MSE en píxeles; devuelve `loss.item()`. Luego corra el entrenamiento dado: la PSNR de validación debe mejorar al menos 5 dB respecto de la red sin entrenar.

Tarea 7 Paso de SRGAN [4 pts]

`paso_srgan(G, D, optG, optD, lr, hr, perdida_contenido, peso_adv)`: un paso de D con `sr.detach()` (real $= 1$, generada $= 0$) y luego un paso de G con

$$\ell^{SR} = \ell_{\text{contenido}} + 10^{-3} \cdot \text{BCE}\big(D(G(I^{LR})),\, 1\big) \qquad \text{(pérdida no saturante)}.$$

Devuelve un diccionario con `'D'`, `'contenido'`, `'adversarial'`, `'D(x)'` y `'D(G(z))'`. Una de las pruebas verifica que, con `peso_adv=0`, el paso de G sea idéntico a un paso de MSE puro. Después corra el entrenamiento dado (VGG54, G inicializado con su SRResNet) y observe las curvas y el recorte comparativo.

Tarea 8 Interpolación de redes [1.5 pts]

`interpolar_redes(G_a, G_b, alfa)`: copia de G_a con $\theta = (1-\alpha)\theta_a + \alpha\theta_b$ en todos los tensores flotantes del `state_dict`; los enteros (`num_batches_tracked`) se copian de G_a; G_a y G_b no se modifican. El notebook evalúa luego $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$.

Tarea 9 Error temporal y video [1.5 pts]

`error_temporal(cuadros, ref)`: $E_{\text{temp}} = \frac{1}{T-1}\sum_t \text{media}\,\big|(\hat{I}_t - \hat{I}_{t-1}) - (I_t - I_{t-1})\big|$, en `float32` (cuidado con el desbordamiento de `uint8`). La celda siguiente genera el video lado a lado bicúbica | SRResNet | SRGAN | HR y reporta E_{temp} de cada método.

Errores frecuentes

- Olvidar `.detach()` en el paso de D: PyTorch fallará al hacer el segundo `backward` sobre el mismo grafo.
- Usar la pérdida saturante $\log(1 - D)$ para G, o etiquetas invertidas.

- Olvidar `optimizer.zero_grad()`: los gradientes se acumulan entre pasos.
- Restar imágenes `uint8` directamente: $3 - 5 = 254$.

3. Plan sugerido

Bloque	Tiempo	Comentario
Preparación	10 min	descarga de Flowers-102 y banco de recortes
Tareas 1–2	20 min	tensores e inicialización
Tareas 3–5	35 min	arquitecturas y pérdida VGG
Tarea 6	10 min	+ entrenamiento (≈3 min)
Tarea 7	20 min	+ entrenamiento (≈5 min, en paralelo con 8–9)
Tareas 8–9	20 min	interpolación, video
Revisión final	5 min	*Reiniciar y ejecutar todo*
Total	**120 min**	

Rúbrica

Tarea	Criterio	Puntos
1–5	Cada prueba `verificar_k()` que pasa recibe el puntaje completo de la tarea; si falla, crédito parcial según el código	10
6	Prueba aprobada y entrenamiento con mejora de PSNR ≥ 5 dB	2
7	Prueba aprobada, entrenamiento completo, curvas y recorte comparativo visibles	4
8–9	Pruebas aprobadas; tabla de α y video generados	3
—	El notebook corre de principio a fin sin errores (*Reiniciar y ejecutar todo*); código propio, legible y comentado	1
	Total	**20**

Qué entregar

El notebook `Lab12b_SRGAN_Programacion.ipynb` ejecutado completo, **con todas las salidas**, incluida la celda final *Resumen de verificación* con las 9 tareas en ✓, y el archivo `lab12b_video.mp4`.

Honestidad académica

Pueden usar asistentes de IA para entender mensajes de error, pero el código de cada tarea debe ser propio. Modificar o eliminar una celda de verificación anula el puntaje de esa tarea.